Observabilidad

Ingeniería de Sistemas

Universidad Austral — 2026

Son las 3 AM

Estás durmiendo. Suena el celular.

El cliente: **«no anda»**.

Los usuarios dicen que sus *snippets* **desaparecieron**.

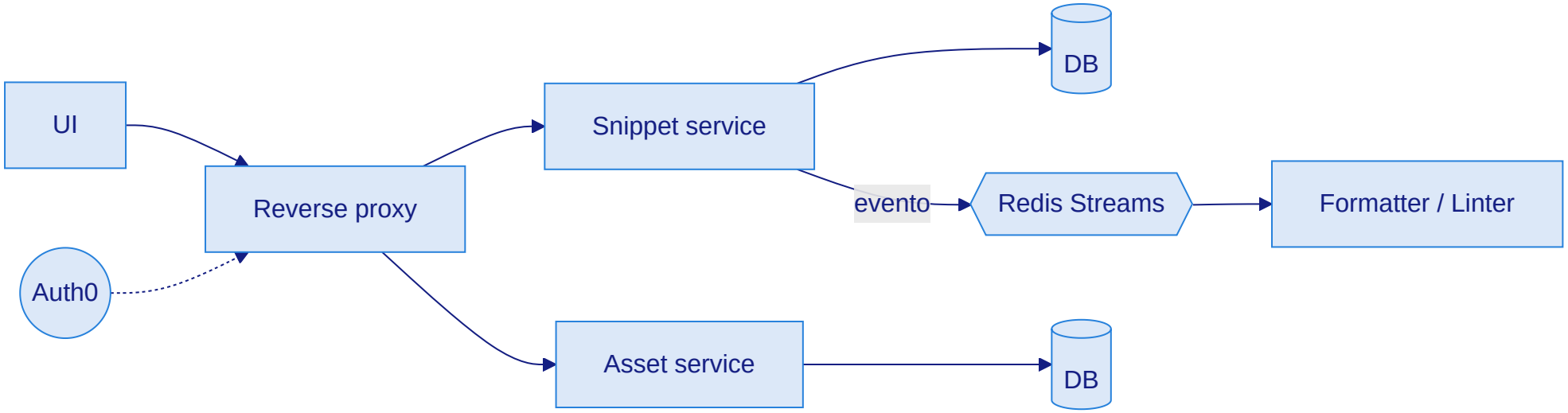
¿Qué hacen?

¿Cómo saben qué parte falla?

Dibujen el Snippet Searcher

De memoria, en dos minutos

Lo que tenemos

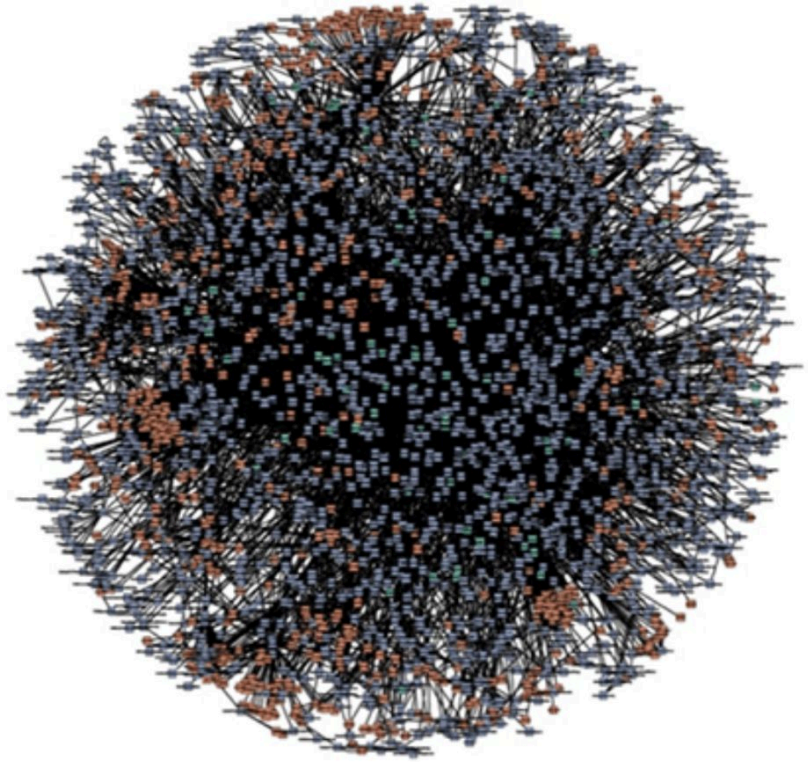


«Mis *snippets* desaparecieron»

	Paso del *request*	Preguntas que nos podemos hacer
1	**UI**	¿Pidió la lista? ¿Qué respondió: error, vacío o datos viejos? ¿Pasa en otro navegador?
2	***Proxy* y Auth0**	¿Es el mismo usuario de ayer? ¿Cambió el `sub` del *token*? ¿Se está filtrando por otro dueño?
3	**Snippet service**	¿Llegó el *request*? ¿Qué consultó? ¿Respondió 200 con una lista vacía?
4	**Asset service**	¿Está arriba? ¿Responde vacío o con error? ¿Se escribió algo hoy?
5	**Base de datos**	¿Siguen los datos? ¿Se reinició el *container*? ¿Estaba el volumen? ¿Cuántas filas hay hoy contra ayer?
6	**Redis y Formatter**	¿Algún proceso asincrónico pudo pisar o borrar datos? ¿Hay mensajes con error?
	En general	¿A todos o a algunos? ¿Desde cuándo? ¿Hubo un *deploy*? ¿`dev` o `prod`?

Cada pregunta es una hipótesis. ¿Cómo las descartamos sin entrar a cada VM?

Si cuesta con 7 componentes...



amazon.com



NETFLIX

...imaginen sistemas más grandes.

Hoy, para contestar esas preguntas

1. Entrar por SSH a la VM de `prod` (y a la de `dev`, por las dudas)
2. Correr `docker logs` en cada *container*, uno por uno
3. Cruzar a mano qué pasó primero, porque cada *log* tiene su formato y su hora
4. Y para «¿cuántas filas había ayer?»: **no hay de dónde sacarlo**

Con un proceso alcanzaba. **Con 7 componentes y 2 ambientes, no.**

Nuestras herramientas para entender

Herramienta	Qué es	Pregunta que responde
Logs	Un evento que ocurrió	¿Qué pasó?
Métricas	Una medición de la salud del sistema	¿Cómo está?
Traces	El camino de un *request* entre servicios	¿Dónde pasó?
Synthetics	Tests automatizados contra producción	¿Funciona lo que le importa al usuario?

Cada una ve **una parte distinta** de la misma falla.

Logs

Un *log* es un evento que ocurrió

Lo que hay hoy: el *log* del *proxy*

```
[02/Nov/2026:18:41:07 -0300] "GET /api/snippets HTTP/1.1" 200 4821
[02/Nov/2026:18:43:52 -0300] "POST /api/snippets HTTP/1.1" 201 312
[03/Nov/2026:03:02:15 -0300] "GET /api/snippets HTTP/1.1" 200 2
[03/Nov/2026:03:02:19 -0300] "GET /api/snippets HTTP/1.1" 200 2
[03/Nov/2026:03:05:40 -0300] "GET /api/snippets HTTP/1.1" 200 2
```

Todo responde 200. Pero la respuesta pasó de **4821 bytes a 2**: `[]`.

El *log* dice **qué** pasó. **No dice por qué.**

¿Qué vale la pena loguear?

- **Cambios de estado** — se creó, se borró o se compartió un *snippet*, y quién lo hizo
- **Resultados** — «se listaron 0 *snippets* de ana»
- **Errores, con contexto** — qué se intentaba hacer y sobre qué
- **Llamadas externas** — a quién, qué respondió y cuánto tardó
- **Nunca** — contraseñas, *tokens* ni secretos

Un error es lo que ya se rompió. **Lo difícil es lo que se rompió sin avisar.**

Para encontrar la línea

Texto libre

```
2026-11-03 03:12:44 INFO Se listaron 0 snippets de ana@mail.com
```

Estructurado (JSON)

```
{"ts":"2026-11-03T03:12:44Z","level":"INFO","service":"snippet-service",
 "msg":"snippets listed","owner":"ana","count":0,"requestId":"7f3a-91c2"}
```

Uno se *grepea*. El otro **se filtra por campo**, y con el `requestId` seguís la operación entera entre servicios.

MDC: el contexto, aparte del mensaje

Sin *MDC*, cada línea mezcla lo que pasó con quién y dónde:

```
"usuario ana, request 7f3a-91c2: snippets listed, count=0"
```

Con *MDC*, el mensaje cuenta solo lo que pasó, y el contexto va aparte:

```
mensaje:   snippets listed, count=0
contexto:  requestId=7f3a-91c2 · user=ana · service=snippet-service
```

- El contexto se carga **una vez**, cuando entra el *request*
- **Todas** las líneas lo llevan, sin que nadie lo escriba

MDC es una convención de Java / Kotlin (SLF4J, Logback), pero la idea vale para cualquier lenguaje.

Guardar los *logs* no es trivial

Cada servicio escribe sus *logs* en su propia VM. Y guardarlos tiene problemas:

- **Volumen** — crece sin parar y ocupa disco
- **Rotación y retención** — ¿cuánto guardamos? ¿Qué se borra y cuándo?
- **Búsqueda** — encontrar una línea entre millones se vuelve lento
- **Costo** — guardar e indexar más *logs* cuesta más
- **Pérdida** — si el *container* muere, ¿se llevan los *logs* con él?

Con un servicio se maneja. **Con muchos, se multiplica.**

Entonces, hay que centralizarlos

Con 6 o 7 servicios en producción, o más, los *logs* **no pueden vivir en cada servidor**.

- Un solo lugar, con un solo formato
- Una sola búsqueda para todos los servicios
- Se sigue un *request* de punta a punta

Lo resuelve una herramienta: Amazon CloudWatch, Datadog, Sumo Logic, Papertrail, New Relic… **Nosotros usamos New Relic.**

Métricas

Una medición de la salud del sistema

¿Qué es una métrica?

Un **número que se mide cada cierto tiempo**, con un nombre y etiquetas.

```
http.server.requests   1532   service=snippet-service  status=200   03:00–03:01
http.server.requests     41   service=snippet-service  status=500   03:00–03:01
```

- Un *log* es **un** evento. Una métrica **resume miles** en un número
- Es barata: se guarda mucho tiempo y se grafica fácil
- Pierde el detalle: dice **cuántos** fallaron, no **cuál**

Las clásicas

- **Tráfico entrante** — *requests* por segundo
- **Duración de *requests*** — tiempo de respuesta
- **Memoria y CPU**
- ***Uptime***

¿Qué medir? Las cuatro señales

Señal	Qué mide	En el Snippet Searcher
Latency	Cuánto tarda	Tiempo del `/format`
Traffic	Cuánta demanda hay	*Requests* por segundo
Errors	Cuánto falla	Respuestas 5xx
Saturation	Cuán lleno está	Largo del *stream* de Redis, *lag* del *consumer*

El promedio miente

100 *requests*: **98 tardan 50 ms** y **2 tardan 5 s**.

Medida	Resultado
Promedio	**149 ms** — parece sano
Percentil 99 (p99)	**5000 ms** — alguien esperó 5 segundos

Para latencia, **mirá percentiles**, no promedios.



Traces

El camino de **un** *request*

¿Qué es un *trace*?

El recorrido de **un** *request* por todos los servicios que toca. Cada tramo es un ***span***: qué servicio, qué hizo y cuánto tardó.

```
trace 7f3a-91c2                                          1200 ms
└─ proxy            GET /api/snippets                    1190 ms
   └─ snippet-service  listar snippets                   1150 ms
      ├─ base de datos   SELECT                            40 ms
      └─ asset-service   GET /assets                     1000 ms
```

¿Dónde se fue el tiempo? **En el Asset service.**

Synthetics

Tests automatizados contra producción

Monitoreo sintético

Correr **un subconjunto de los tests automatizados contra producción**, cada cierto tiempo.

Los resultados van al sistema de monitoreo, que **alerta si algo falla**.

Combina *testing* automatizado con monitoreo: detecta **requisitos de negocio rotos en producción**.

Ejemplo: un *check* por minuto

1. Crear un *snippet*
2. Leerlo
3. Borrarlo

Si falla, **algo del flujo está roto** aunque todos los servicios estén «arriba».

Otro nivel

¿Cuándo está «bien»? ¿Cuándo nos avisan?

SLI, SLO y SLA

- **SLI** — *Service Level Indicator*
  La métrica que mide el servicio: «% de lecturas que responden bien en menos de 300 ms»
- **SLO** — *Service Level Objective*
  El objetivo para esa métrica: «99,5 % en los últimos 30 días»
- **SLA** — *Service Level Agreement*
  Lo que se le promete al cliente, **con consecuencias**: «99 %, o devolvemos parte de lo que pagó»

Disponibilidad (*availability*)

El SLI más común: **¿qué parte del tiempo, o de los *requests*, el sistema anduvo bien?**

Por tiempo	Por *requests*
tiempo arriba / tiempo total	*requests* exitosos / *requests* totales
«estuvo caído 2 h este mes»	«falló el 2,6 % de los pedidos»
un minuto a las 3 AM pesa igual que uno en el pico	pesa más donde hay más gente

Antes de medirla hay que decidir **qué cuenta como éxito**: ¿un 200? ¿En menos de 300 ms? ¿Con los datos correctos?

¿Cuánto es un 99 %?

SLO en 30 días	Se puede fallar
99 %	7 h 12 min
99,5 %	3 h 36 min
99,9 %	43 min
99,99 %	4 min 19 s

Lo que sobra es el ***error budget***: cuánto se puede fallar antes de romper el SLO.

Error budget: fallar también se presupuesta

Ningún sistema está al 100 %, y **no hay que prometerlo**. Lo que queda entre el SLO y el 100 % es un presupuesto: el equipo decide **en qué gastarlo**.

Con un SLO de 99,5 % en 30 días hay **3 h 36 min** para gastar:

En qué se fue	¿A propósito?	
Migrar la base a la versión nueva	sí, planificado	30 min
Probar un *feature* nuevo y revertirlo	sí, un riesgo elegido	40 min
Redis caído: el *linting* no corrió	no, un incidente	30 min
Queda		**1 h 56 min**

¿Para qué sirve?

Para decidir con un número, no con opiniones:

- **Queda presupuesto** → se puede arriesgar: *deployar* algo nuevo, experimentar
- **Se gastó** → se frenan los *releases* y se prioriza la estabilidad hasta recuperarlo

Desarrollo quiere sacar cosas nuevas; operaciones, que nada se rompa.
El *error budget* es **el mismo número para los dos**.

¿Hay horarios de uso?

Fallar a las 3 AM no cuesta lo mismo que fallar en el pico, **si es que hay pico**.

- **Mírenlo en las métricas**: *requests* por hora y por día de la semana
- **Pregúntenle al negocio**: ¿hay fechas fuertes? Inscripciones, entregas, fin de mes
- **Úsenlo**: *deploys* y mantenimiento en las horas de menos uso, capacidad para el pico, alertas que tengan en cuenta la hora
- Si el mantenimiento fuera de horario gasta presupuesto **depende de lo acordado**. Mejor todavía: no necesitar *ventanas de mantenimiento**

* *Ventana de mantenimiento*: un horario acordado y avisado de antemano en el que el sistema puede no estar disponible, para hacer tareas planificadas como migraciones o actualizaciones.

¿Nuestro SLI hubiera visto los *snippets* desaparecidos?

«% de lecturas que responden **200** en menos de 300 ms»: **100 %**.

El SLO se cumplía. **Los usuarios no tenían sus datos.**

> *"Start by thinking about (or finding out!) what your users care about, not what you can measure."*
> *— Site Reliability Engineering, cap. 4*

Alertas

Una **notificación de una medición anormal**. Pueden basarse en *logs* o en métricas.

- Memoria, CPU o disco de la base al límite
- % de errores de un servicio por encima de un umbral
- Uso anormal de memoria o CPU

Una buena alerta

- **Accionable** — alguien sabe qué hacer cuando suena
- **Urgente** — no puede esperar a mañana
- **Con dueño** — hay una persona responsable

Si todo alerta, **nada alerta**. Alertá cuando estás por **romper el SLO**, no cuando algo «se ve raro».

Para seguir leyendo

Domain-Oriented Observability — Pete Hodgson, publicado en el sitio de Martin Fowler (2019)

https://martinfowler.com/articles/domain-oriented-observability.html

Cómo instrumentar **eventos del negocio** (un descuento aplicado, un pago fallido) en lugar de solo métricas técnicas, sin ensuciar la lógica.